

Package ‘fspls2’

July 3, 2026

Type Package

Title A Package to Find Minimal Transcriptional Signatures

Version 0.1.0

Date 2026-06-25

Author Lachlan Coin [aut, cre]

Imports R6, jsonlite, Matrix, glmnet, tidyr, MASS, ggplot2, pROC,
binom, confintr, DBI, RSQLite, methods, tibble, ggrepel,
RColorBrewer, parallel

Depends R (>= 4.1.0)

Maintainer Lachlan Coin <l.coin@imb.uq.edu.au>

Description Identifies minimal gene signatures from multiomic data. Based on methods described in Rawlinson et al. (2025) <[doi:10.1371/journal.pdig.0000780](https://doi.org/10.1371/journal.pdig.0000780)>.

License GPL-2

LazyLoad yes

NeedsCompilation no

Repository CRAN

Date/Publication 2026-06-25 13:50:02 UTC

Config/roxygen2/version 8.0.0

Encoding UTF-8

Suggests knitr, rmarkdown, testthat

VignetteBuilder knitr

Contents

analysisBase	2
analysisEnv	3
check_flags	5
dataH	5
getFamily	10
getYTransform	11
plotEval	11

Index**14**

analysisBase	<i>Analysis Base Class</i>
--------------	----------------------------

Description

A class that encapsulates a dataset holder and/or analysis of datasets

Details

Available methods:

- `new()` - Create a new instance
- `name()` - Get dataset name
- `clone()` - Clone the object

Methods**Public methods:**

- [analysisBase\\$new\(\)](#)
- [analysisBase\\$name\(\)](#)
- [analysisBase\\$clone\(\)](#)

`analysisBase$new()`: Create a new instance of base class

Usage:

```
analysisBase$new(nme, dims = NULL, flags = list(), dbDir = tempdir())
```

Arguments:

`nme` name

`dims` dims

`flags` a list object specifying options

`dbDir` location for database to be written, default to tempdir

`analysisBase$name()`: get dataset name;

Usage:

```
analysisBase$name()
```

Returns: assigned name of dataset

`analysisBase$clone()`: The objects of this class are cloneable with this method.

Usage:

```
analysisBase$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

analysisEnv	<i>Analysis Class</i>
-------------	-----------------------

Description

A class that encapsulates analysis of one or more datasets

Details

Available methods:

- `new()` - Create a new instance
- `name()` - Get dataset name
- `clone()` - Clone the object

Super class

`analysisBase` -> `analysisEnv`

Methods

Public methods:

- `analysisEnv$new()`
- `analysisEnv$getTodo()`
- `analysisEnv$clear_db()`
- `analysisEnv$savePvalsAndNextVars()`
- `analysisEnv$select()`
- `analysisEnv$clone()`

`analysisEnv$new()`: Create a new instance

Usage:

```
analysisEnv$new(flags = list(), dbDir = tempdir())
```

Arguments:

`flags` a list object specifying options

`dbDir` location for database to be written. If NULL then no DB Get todo

`analysisEnv$getTodo()`:

Usage:

```
analysisEnv$getTodo(flags, phens)
```

Arguments:

`flags` list of options

`phens` phenotypes

Returns: object outlining what is left to do for next iteration Clear the databases. Only necessary if using a database and re-running analyses

analysisEnv\$clear_db():

Usage:

```
analysisEnv$clear_db(drop = F, exclude = "vars", datasH = NULL)
```

Arguments:

drop completely drop tables and start from scratch

exclude tables to exclude from clearing

datasH list of dataH objects to clear Get the next vars in iteration. Internal function

analysisEnv\$savePvalsAndNextVars():

Usage:

```
analysisEnv$savePvalsAndNextVars(
  flags,
  phens,
  vars_l_todo,
  comb2,
  data_nme,
  k1
)
```

Arguments:

flags list of options

phens phenotypes

vars_l_todo vars_l_todo

comb2 results

data_nme name of dataset

k1 which repetition

Returns: object outlining what is left to do main function for variable selection

analysisEnv\$select():

Usage:

```
analysisEnv$select(
  datasH,
  flags,
  transform_y,
  phens = datasH[[1]]$pheno()$all,
  useDB = F
)
```

Arguments:

datasH a list of dataH objects

flags list of options

transform_y transformation object

phens list of phenotypes

useDB boolean to indicate if results should be saved to database

analysisEnv\$clone(): The objects of this class are cloneable with this method.

Usage:

```
analysisEnv$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

check_flags

Plot the results from evaluateAll

Description

Plot the results from evaluateAll

Usage

```
check_flags(flags)
```

Arguments

flags a list of options

Value

corrected flags

dataH

Data holder class

Description

A class that encapsulates a dataset holder

Details

Available methods:

- new() - Create a new instance
- name() - Get dataset name
- clone() - Clone the object

Super class

`analysisBase` -> dataH

Methods

Public methods:

- `dataH$new()`
- `dataH$multiAnglesAndPv()`
- `dataH$clear_db()`
- `dataH$data_types()`
- `dataH$split()`
- `dataH$pheno()`
- `dataH$dims()`
- `dataH$nreps()`
- `dataH$update()`
- `dataH$select()`
- `dataH$extractPredictions()`
- `dataH$plotData()`
- `dataH$getProjectedData()`
- `dataH$getVariance()`
- `dataH$makeAllModels()`
- `dataH$evaluateAllModels()`
- `dataH$clone()`

`dataH$new()`: Create a new instance

Usage:

```
dataH$new(
  data,
  y,
  nme,
  flags,
  transform_y = getYTransform(pow = 1, n_random = 1, perm = F),
  family = getFamily(y),
  dbDir = tempdir()
)
```

Arguments:

`data` a list of matrices (of different modalities), with columns as variables and rows as samples

`y` phenotype matrix, with columns as outcomes and rows as samples

`nme` The name of the data object.

`flags` list of options, described in the vignette

`transform_y` a transformation object from the function `getYTransformation`

`family` the statistical family of phenotype `y`, can be calculated by `getFamily`

`dbDir` dir for database to store results to speed up re-runs. Can be NULL, if not required

`dataH$multiAnglesAndPv()`: Calculate the angles and `pv` across multiple phenotypes. This is an internal function and should not need to be called by user

Usage:

```
dataH$multiAnglesAndPv(comb20, phens1, k1, flags, expt_id, vars_l_todo)
```

Arguments:

comb20 values from previous iteration

phens1 phenotypes being used

k1 k1 is the current fold

flags list of options

expt_id and ID for the experiment (can be 0)

vars_l_todo the variables under consideration clear database. This is only used if you want to clear the database. Internal function

```
dataH$clear_db():
```

Usage:

```
dataH$clear_db(drop = FALSE, exclude = "vars")
```

Arguments:

drop drop the tables instead of clear

exclude which tables not to clear get the data types

```
dataH$data_types():
```

Usage:

```
dataH$data_types()
```

Returns: which data types are in the dataset

```
dataH$split(): split dataset into smaller datasets
```

Usage:

```
dataH$split(proportions = c(0.5, 0.5))
```

Arguments:

proportions what proportions to split into

Returns: a list of dataH objects with data partiioned according to proportions

```
dataH$pheno(): get the phenotyeps
```

Usage:

```
dataH$pheno()
```

Returns: phenotypes

```
dataH$dims(): get the dimensions
```

Usage:

```
dataH$dims()
```

Returns: dimensions get the nreps

```
dataH$nreps():
```

Usage:

```
dataH$nreps()
```

Returns: nreps

`dataH$update()`: update the phenotypes without remaking the entire object. You can provide many phenotypes in the initialisation stage, but only consider a subset in model fitting stage in this way.

Usage:

```
dataH$update(phens, flags, transform_y = fromJSON(flags$transform_y))
```

Arguments:

phens phenotypes

flags list of options

transform_y transformation object

`dataH$select()`: main function for variable selection

Usage:

```
dataH$select(analysis, phens, flags, transform_y, useDB = F)
```

Arguments:

analysis an analysisEnv object

phens list of phenotyps

flags list of options

transform_y transformation object

useDB boolean to indicate if results should be saved to database

`dataH$extractPredictions()`: extract the predictions for the fitted models

Usage:

```
dataH$extractPredictions(
  all_modelsh,
  phens = all_modelsh$phens,
  flags = all_modelsh$flags,
  CV = FALSE,
  liab = T
)
```

Arguments:

all_modelsh fitted models from makeAllModels

phens list of phenotyps

flags list of options

CV cross validation predictions

liab return liability score, or probability (for binomial, ordinal multinomial)

Returns: a table with results

`dataH$plotData()`: plot all data for selected variables

Usage:


```
dataH$plotData(  
  vars_all,  
  phens = vars_all$phens,  
  all_types = FALSE,  
  transform_x = NULL,  
  violin = FALSE,  
  assoc = F  
)
```

Arguments:

vars_all variables

phens list of phenotyps

all_types use all types?

transform_x which transform to use on x

violin violin plots

assoc use association

Returns: a table with results

dataH\$getProjectedData(): get data after projection

Usage:

```
dataH$getProjectedData(varnames)
```

Arguments:

varnames varnames

Returns: projected data after projecting out varnames

dataH\$getVariance(): get variance of data

Usage:

```
dataH$getVariance(varnames)
```

Arguments:

varnames varnames

Returns: variance

dataH\$makeAllModels(): fit models based on variables

Usage:

```
dataH$makeAllModels(  
  vars_all,  
  phens = vars_all[[1]]$phens,  
  flags = vars_all[[1]]$flags,  
  transform_y = fromJSON(flags$transform_y),  
  useDB = F  
)
```

Arguments:

vars_all list of variables selected by select method

phens list of phenotypes

flags flags
 transform_y an object to determine the y transformaton
 useDB whether to use the DB to save the reuslt
Returns: fitted models

dataH\$evaluateAllModels(): evaluate the fit of models

Usage:

```
dataH$evaluateAllModels(
  all_modelsh,
  phens = all_modelsh$phens,
  flags = all_modelsh$flags,
  useDB = F
)
```

Arguments:

all_modelsh models fitted from makeAllModels
 phens list of phenotypes
 flags flags
 useDB whether to save results to DB

Returns: evaluation of models

dataH\$clone(): The objects of this class are cloneable with this method.

Usage:

```
dataH$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

getFamily

Infer the statistical families of the phenotypes

Description

Infer the statistical families of the phenotypes

Usage

```
getFamily(y_mat, max_ordinal = getOption("max_ordinal", 10))
```

Arguments

y_mat	a matrix of phenotypes, with columns as variables and rows as samples
max_ordinal	maximum number of values before we consider an ordinal value as a continuous value

Value

a list with the statistical families detected

getYTransform	<i>Get object for transforming the y variables</i>
---------------	--

Description

Get object for transforming the y variables

Usage

```
getYTransform(  
  pows = c(1),  
  offset = 0,  
  n_random = 1,  
  perm = F,  
  norm = 1,  
  CHECK = F  
)
```

Arguments

pows	power to raise to
offset	offset to subtract
n_random	how many random variables
perm	whether random is permutation
norm	rescaling factor
CHECK	check whether inverse function works

Value

transformation object

plotEval	<i>Plot the results from evaluateAll</i>
----------	--

Description

Plot the results from evaluateAll

Usage

```

plotEval(
  eval3,
  shape_color = c("pheno", "subpheno"),
  shape = shape_color,
  text = "variable",
  dotsize = "nsamps",
  color = shape_color,
  linetype = shape,
  showranges = TRUE,
  txtsize = 1,
  logy = F,
  legend = F,
  sep_by = "",
  scales = "free",
  point = TRUE,
  line = TRUE,
  labelsiz = 2,
  grid0 = c("cohort", "measure"),
  grid1 = "cv_full",
  title = "",
  title1 = ""
)

```

Arguments

eval3	a tibble or data frame from evaluateAll method from dataH
shape_color	a list of column names to encode as both shape and color
shape	defaults to shape_color but can be specified separately
text	a list of column names to include as text labels
dotsize	a numerical column to encode dotsize, or a number for constant sizes
color	defaults to shape_color but can be specified separately
linetype	column controlling the type of line
showranges	whether to show 95% CI as ribbon
txtsize	size of text
logy	whether to log y axis
legend	whether to show a legend
sep_by	list of column names to separate into diff plots
scales	used in faceting can be free or fixed
point	whether to show dots
line	whether to include line
labelsiz	the size of the text labels
grid0	how to facet on y

plotEval

13

<code>grid1</code>	how to facet on x
<code>title</code>	the title
<code>title1</code>	columns to include in the title

Value

ggplot2 plot

Index

analysisBase, [2](#), [3](#), [5](#)

analysisEnv, [3](#)

check_flags, [5](#)

dataH, [5](#)

getFamily, [10](#)

getYTransform, [11](#)

plotEval, [11](#)